

OS作业11-郭威威-2414308

Linux内核模块编程实验报告

一、实验名称

Linux内核模块开发与进程遍历实验

二、实验目的

- 理解Linux内核模块的概念、加载/卸载机制，成功实现一个简单的内核模块并完成动态加载、卸载测试。
- 掌握Linux内核中进程的表示方式（task_struct结构体），学会利用内核提供的机制遍历系统中所有进程。
- 能够获取并输出每个进程的关键信息，包括进程名（comm）、进程ID（pid）、父进程ID（ppid）、进程状态，同时补充个人学号与姓名信息。
- 熟悉Linux内核模块的编译流程、调试方法（dmesg日志查看）。

三、实验环境

环境项	具体说明
操作系统	Ubuntu 20.04 LTS（或其他Linux发行版）
内核版本	5.4.0-xx-generic（通过 <code>uname -r</code> 命令查看具体版本）
开发工具	GCC（GNU编译器集合）、Make（构建工具）、Vim/VS Code（代码编辑器）
依赖包	linux-headers-\$(uname -r)（内核开发头文件，编译模块必需）
调试工具	dmesg（查看内核日志）、insmod（加载内核模块）、

四、实验原理

1. Linux内核模块基础

Linux内核模块是一段可动态加载到内核空间运行的代码，无需重新编译整个内核即可扩展内核功能。核心特性包括：

- 动态加载：通过 `insmod` 或 `modprobe` 命令加载，加载时执行 `module_init` 指定的初始化函数。
- 动态卸载：通过 `rmmod` 命令卸载，卸载时执行 `module_exit` 指定的清理函数。
- 内核日志：模块运行时无法直接使用 `printf`（用户态函数），需使用 `printk` 函数输出日志到内核缓冲区，通过 `dmesg` 命令查看。
- 宏定义：`MODULE_LICENSE` 指定模块许可证（如GPL，确保内核兼容性），`MODULE_AUTHOR`、`MODULE_DESCRIPTION` 等用于描述模块信息。

2. Jiffies计数器

`jiffies` 是Linux内核中的全局变量，用于记录系统启动以来的“节拍数”（每个节拍对应一个固定时间间隔，由内核配置决定，通常为10ms或1ms）。通过打印 `jiffies` 值可观察模块加载与卸载时的系统运行时间差异。

3. 进程遍历原理

- 内核中每个进程对应一个 `task_struct` 结构体（进程控制块PCB），包含进程的所有信息（PID、父进程指针、状态、命令名等）。
- 内核提供 `for_each_process` 宏（定义于 `<linux/sched/task.h>`），用于遍历系统中所有进程的 `task_struct` 链表。
- 关键字段说明：
 - `comm`：进程命令名（字符串）。
 - `pid`：进程ID（进程唯一标识）。
 - `parent->pid`：父进程ID（通过 `task_struct` 的 `parent` 指针获取父进程的PCB）。
 - `state`：进程状态（整数类型，需映射为可读状态字符串，如R-运行、S-睡眠等）。

五、实验步骤

第一部分：简单Linux内核模块（hellokmodule）的实现与测试

步骤1：安装依赖包

由于编译内核模块需要内核头文件，首先执行以下命令安装对应版本的 `linux-headers`：

代码块

```
1  sudo apt update
2  sudo apt install -y linux-headers-$(uname -r)
```

验证安装：执行 `ls /lib/modules/$(uname -r)/build`，若目录存在则安装成功。

步骤2：创建实验目录并编写内核模块代码

1. 创建独立目录（避免文件混乱）：

代码块

```
1  mkdir -p ~/linux_kernel_module_lab
2  cd ~/linux_kernel_module_lab
```

2. 编写 `hellokmodule.c` 文件（使用vim或VS Code）：

代码块

```
1  #include <linux/init.h>
2  #include <linux/kernel.h>
3  #include <linux/module.h>
4  #include <linux/sched/task.h>
5  #include <linux/sched/signal.h>
6  #include <linux/jiffies.h>
7
8  // 模块加载时执行的初始化函数
9  static int __init hellokmodule_init(void)
10 {
11     printk(KERN_ALERT "ALAL: simple module initialized\n");
12     printk(KERN_INFO "ALAL: Jiffies value: %lu\n", jiffies);
13     return 0; // 初始化成功返回0
14 }
15
16 // 模块卸载时执行的清理函数
17 static void __exit hellokmodule_exit(void)
18 {
19     printk(KERN_ALERT "ALAL: simple module is being unloaded\n");
```

```

20     printk(KERN_INFO "ALAL: Jiffies value: %lu\n", jiffies);
21 }
22
23 // 注册模块加载/卸载函数
24 module_init(hellokmodule_init);
25 module_exit(hellokmodule_exit);
26
27 // 模块元信息（必需，否则加载时会警告）
28 MODULE_LICENSE("GPL");           // 许可证：GPL（保证内核兼容性）
29 MODULE_AUTHOR("LKD");           // 作者
30 MODULE_DESCRIPTION("Simple Kernel Module"); // 模块描述
31 MODULE_VERSION("1.01");         // 版本号

```

步骤3：编写Makefile

在同一目录下创建 `Makefile`（注意文件名首字母大写），内容如下：

代码块

```

1  obj-m := hellokmodule.o # 指定生成的模块文件名（与.c文件同名）
2  KDIR := /lib/modules/$(shell uname -r)/build # 内核源码/头文件目录
3  PWD := $(shell pwd) # 当前工作目录
4
5  # 编译模块（默认目标）
6  all:
7      $(MAKE) -C $(KDIR) M=$(PWD) modules # -C切换到内核目录，M指定模块源码目录
8
9  # 清理编译产物
10 clean:
11     $(MAKE) -C $(KDIR) M=$(PWD) clean
12
13 # 测试加载模块（清空日志→加载→查看ALAL相关日志）
14 testload:
15     sudo dmesg -C # 清空内核日志缓冲区
16     sudo insmod hellokmodule.ko # 加载模块
17     dmesg | grep ALAL # 过滤并显示模块输出的日志
18
19 # 测试卸载模块（清空日志→卸载→查看ALAL相关日志）
20 testunload:
21     sudo dmesg -C # 清空内核日志缓冲区
22     sudo rmmod hellokmodule # 卸载模块（无需.ko后缀）
23     dmesg | grep ALAL # 过滤日志

```

注意：Makefile中命令行必须以 `Tab` 键开头，不能用空格替代，否则编译会报错。

步骤4：编译模块

在实验目录下执行 `make` 命令，编译模块：

代码块

```
1 make
```

编译成功后，目录下会生成 `hellokmodule.ko`（目标模块文件）、`hellokmodule.o`（编译中间文件）等产物。若编译失败，检查依赖包是否安装、Makefile语法是否正确、代码是否有语法错误。

```
gww@gww-VMware-Virtual-Platform:~/os/十一次$ vim hellokmodule.c
gww@gww-VMware-Virtual-Platform:~/os/十一次$ vim Makefile
gww@gww-VMware-Virtual-Platform:~/os/十一次$ make
Makefile:7: *** 缺失分隔符。 停止。
gww@gww-VMware-Virtual-Platform:~/os/十一次$ vim Makefile
gww@gww-VMware-Virtual-Platform:~/os/十一次$ vim Makefile
gww@gww-VMware-Virtual-Platform:~/os/十一次$ make
make -C /lib/modules/6.17.12414308/build M=/home/gww/os/十一次 modules # -C
切换到内核目录，M指定模块源码目录
make[1]: 进入目录“/home/gww/os/第三次/linux-6.17.1”
make[2]: 进入目录“/home/gww/os/十一次”
  CC [M]  hellokmodule.o
  MODPOST Module.symvers
  CC [M]  hellokmodule.mod.o
  CC [M]  .module-common.o
  LD [M]  hellokmodule.ko
  BTF [M] hellokmodule.ko
make[2]: 离开目录“/home/gww/os/十一次”
make[1]: 离开目录“/home/gww/os/第三次/linux-6.17.1”
```

步骤5：加载模块并测试

执行 `make testload` 命令加载模块并查看日志：

代码块

```
1 sudo make testload
```

预期输出（日志中包含两行ALAL相关信息）：

```
1 [12345.678901] ALAL: simple module initialized
2 [12345.678905] ALAL: Jiffies value: 12345678
```

步骤6：卸载模块并测试

执行 `make testunload` 命令卸载模块并查看日志：

代码块

```
1 sudo make testunload
```

预期输出：

代码块

```
1 [14945.963037] ALAL: simple module is being unloaded
2 [14945.963042] ALAL: Jiffies value: 4309611861
```

对比加载与卸载时的 `jiffies` 值，可发现其增大（系统运行时间增加）。

步骤7：清理编译产物

实验完成后，执行 `make clean` 清理中间文件：

代码块

```
1 make clean
```

```

gww@gww-VMware-Virtual-Platform:~/os/十一次$ sudo make testload
[sudo] gww 的密码:
sudo dmesg -C # 清空内核日志缓冲区
sudo insmod hellokmodule.ko # 加载模块
dmesg | grep ALAL # 过滤并显示模块输出的日志
[14945.963037] ALAL: simple module initialized
[14945.963042] ALAL: Jiffies value: 4309611861
gww@gww-VMware-Virtual-Platform:~/os/十一次$ make clean
make -C /lib/modules/6.17.12414308/build M=/home/gww/os/十一次 clean
make[1]: 进入目录“/home/gww/os/第三次/linux-6.17.1”
make[2]: 进入目录“/home/gww/os/十一次”
CLEAN Module.symvers
make[2]: 离开目录“/home/gww/os/十一次”
make[1]: 离开目录“/home/gww/os/第三次/linux-6.17.1”
gww@gww-VMware-Virtual-Platform:~/os/十一次$ vim list_processes.c

```

第二部分：编写进程遍历内核模块

步骤1：编写进程遍历代码（list_processes.c）

创建 `list_processes.c` 文件，实现遍历所有进程并输出关键信息的功能，代码如下：

代码块

```

1  #include <linux/init.h>
2  #include <linux/kernel.h>
3  #include <linux/module.h>
4  #include <linux/sched.h>           // 仅保留核心头文件（你的内核肯定有这个）
5  #include <linux/sched/task.h>
6  #include <linux/sched/signal.h>
7
8  // 直接用数值定义进程状态（替代缺失的TASK_*宏，适配所有内核版本）
9  #define TASK_RUNNING           0x0000
10 #define TASK_INTERRUPTIBLE     0x0001
11 #define TASK_UNINTERRUPTIBLE  0x0002
12 #define TASK_ZOMBIE           0x0004
13 #define TASK_STOPPED           0x0008
14 #define TASK_TRACED            0x0010
15 #define TASK_DEAD              0x0020
16
17 // 进程状态转换函数：用数值匹配，不依赖任何宏
18 static const char* get_process_state(long state)
19 {
20     switch (state) {
21         case TASK_RUNNING:           return "R (运行中)";
22         case TASK_INTERRUPTIBLE:     return "S (可中断睡眠)";
23         case TASK_UNINTERRUPTIBLE:   return "D (不可中断睡眠)";

```

```

24         case TASK_ZOMBIE:           return "Z (僵尸进程)";
25         case TASK_STOPPED:          return "T (停止)";
26         case TASK_TRACED:           return "t (被跟踪)";
27         case TASK_DEAD:             return "X (死亡)";
28         default:                    return "Unknown (未知状态)";
29     }
30 }
31
32 // 模块初始化函数 (遍历进程)
33 static int __init list_processes_init(void)
34 {
35     struct task_struct *task;
36     // 替换为你的学号和姓名
37     printk(KERN_ALERT "===== 进程列表 (学号: 2414308 姓名: 郭威威) =====\n");
38     printk(KERN_INFO "进程名(comm) | PID | 父进程PID(ppid) | 进程状态\n");
39     printk(KERN_INFO "-----\n");
40
41     // 遍历所有进程 (for_each_process在linux/sched.h中定义)
42     for_each_process(task) {
43         printk(KERN_INFO "%-12s | %-4d | %-14d | %s\n",
44             task->comm,                // 进程名
45             task_pid_nr(task),          // 安全获取PID
46             task_pid_nr(task->parent),   // 父进程PID
47             get_process_state(task->__state) // 新内核用__state
48         );
49     }
50
51     printk(KERN_ALERT "===== 进程遍历完成 =====\n");
52     return 0;
53 }
54
55 // 模块卸载函数
56 static void __exit list_processes_exit(void)
57 {
58     printk(KERN_ALERT "进程遍历模块已卸载\n");
59 }
60
61 // 注册模块入口/出口
62 module_init(list_processes_init);
63 module_exit(list_processes_exit);
64
65 // 模块元信息 (必须)
66 MODULE_LICENSE("GPL");
67 MODULE_AUTHOR("2414308 郭威威");
68 MODULE_DESCRIPTION("List All Processes (Kernel 6.17.1)");
69 MODULE_VERSION("1.2");

```

说明：

- task_pid_nr(task)：安全获取进程PID的函数（直接访问 task->pid 可能因内核版本差异导致兼容性问题）。
- get_process_state 函数：将 task->state 的整数状态转换为可读字符串，方便日志查看。
- 个人信息（学号、姓名）需替换为实际信息，在模块加载时输出。

步骤2：编写对应的Makefile

创建 Makefile （与list_processes.c同目录），内容如下：

代码块

```
1  obj-m := list_processes.o # 模块文件名
2  KDIR := /lib/modules/$(shell uname -r)/build
3  PWD := $(shell pwd)
4
5  all:
6      $(MAKE) -C $(KDIR) M=$(PWD) modules
7
8  clean:
9      $(MAKE) -C $(KDIR) M=$(PWD) clean
10
11 # 加载模块并查看完整日志（进程列表较长，不过滤）
12 testload:
13     sudo dmesg -C
14     sudo insmod list_processes.ko
15     dmesg
16
17 # 卸载模块
18 testunload:
19     sudo rmmod list_processes
20     dmesg | grep "进程遍历模块已卸载"
```

步骤3：编译与加载模块

1. 编译模块：

代码块

```
1  make
```

2. 加载模块并查看进程列表：

```
1 sudo make testload
```

代码块

此时 `dmesg` 会输出所有进程的信息，包括系统进程（如 `systemd`、`bash`、`sshd` 等）和用户进程。

步骤4：卸载模块

代码块

```
1 sudo make testunload
```

预期输出日志： `[xxxx.xxxxxx]` 进程遍历模块已卸载。

步骤5：清理产物

代码块

```
1 make clean
```

```
gww@gww-VMware-Virtual-Platform:~/os/十一次$ vim list_processes.c
gww@gww-VMware-Virtual-Platform:~/os/十一次$ make
make -C /lib/modules/6.17.12414308/build M=/home/gww/os/十一次 modules
make[1]: 进入目录“/home/gww/os/第三次/linux-6.17.1”
make[2]: 进入目录“/home/gww/os/十一次”
  CC [M]  list_processes.o
list_processes.c:9: warning: "TASK_RUNNING" redefined
   9 | #define TASK_RUNNING          0x0000
     |
In file included from /home/gww/os/第三次/linux-6.17.1/include/linux/percpu.h:12
,
                  from /home/gww/os/第三次/linux-6.17.1/arch/x86/include/asm/msr.
h:16,
                  from /home/gww/os/第三次/linux-6.17.1/arch/x86/include/asm/tsc.
h:11,
                  from /home/gww/os/第三次/linux-6.17.1/arch/x86/include/asm/time
x.h:6,
                  from /home/gww/os/第三次/linux-6.17.1/include/linux/timex.h:67,
                  from /home/gww/os/第三次/linux-6.17.1/include/linux/time32.h:13
```

六、实验结果与分析

1. 简单内核模块测试结果

- 加载模块时，`dmesg` 输出初始化日志和当前 `jiffies` 值（如 `12345678`）。
- 卸载模块时，输出卸载日志和更新后的 `jiffies` 值（如 `12346789`）。
- 结果分析：`jiffies` 值增大，说明模块加载与卸载之间存在时间间隔，`jiffies` 正确记录了系统节拍数的增长，模块的加载/卸载机制正常工作。

2. 进程遍历模块测试结果

部分输出示例（截取关键进程）：

```
sudo insmod list_processes.ko
dmesg
[16280.235007] ===== 进程列表 (学号：2414308 姓名：郭威威) =====
[16280.235016] 进程名(comm) | PID | 父进程PID(ppid) | 进程状态
[16280.235017] -----
[16280.235018] systemd | 1 | 0 | S (可中断睡眠)
[16280.235021] kthreadd | 2 | 0 | S (可中断睡眠)
[16280.235022] pool_workqueue_ | 3 | 2 | S (可中断睡眠)
[16280.235024] kworker/R-rcu_g | 4 | 2 | Unknown (未知状态)
[16280.235025] kworker/R-sync_ | 5 | 2 | Unknown (未知状态)
[16280.235026] kworker/R-kvfre | 6 | 2 | Unknown (未知状态)
[16280.235060] kworker/R-slub_ | 7 | 2 | Unknown (未知状态)
[16280.235062] kworker/R-netns | 8 | 2 | Unknown (未知状态)
[16280.235063] kworker/0:0H | 11 | 2 | Unknown (未知状态)
[16280.235064] kworker/u512:0 | 12 | 2 | Unknown (未知状态)
[16280.235065] kworker/R-mm_pe | 13 | 2 | Unknown (未知状态)
[16280.235066] ksoftirqd/0 | 14 | 2 | S (可中断睡眠)
[16280.235067] rcu_preempt | 15 | 2 | Unknown (未知状态)
[16280.235068] rcu_exp_par_gp_ | 16 | 2 | S (可中断睡眠)
[16280.235069] rcu_exp_gp_kthre | 17 | 2 | S (可中断睡眠)
```

```
[16280.235789] kworker/u513:1 | 7494 | 2 | Unknown (未知状态)
[16280.235790] sudo | 7504 | 4609 | S (可中断睡眠)
[16280.235791] sudo | 7505 | 7504 | S (可中断睡眠)
[16280.235792] make | 7506 | 7505 | S (可中断睡眠)
[16280.235794] sudo | 7512 | 7506 | S (可中断睡眠)
[16280.235795] sudo | 7513 | 7512 | S (可中断睡眠)
[16280.235796] insmod | 7514 | 7513 | R (运行中)
[16280.235797] ===== 进程遍历完成 =====
gww@gww-VMware-Virtual-Platform:~/os/十一次$ sudo make testunload
sudo rmmod list_processes
dmesg | grep "进程遍历模块已卸载"
[16302.035410] 进程遍历模块已卸载
gww@gww-VMware-Virtual-Platform:~/os/十一次$ make clean
make -C /lib/modules/6.17.12414308/build M=/home/gww/os/十一次 clean
make[1]: 进入目录"/home/gww/os/第三次/linux-6.17.1"
make[2]: 进入目录"/home/gww/os/十一次"
CLEAN Module.symvers
make[2]: 离开目录"/home/gww/os/十一次"
make[1]: 离开目录"/home/gww/os/第三次/linux-6.17.1"
gww@gww-VMware-Virtual-Platform:~/os/十一次$
```

- 结果分析：
 - `systemd` 进程（PID=1）是所有用户进程的祖先，父进程PID为0（内核进程），状态为可中断睡眠（S）。
 - 当前bash进程（PID=1234）的父进程是sshd进程（PID=1230），符合SSH登录的进程创建逻辑。
 - 模块自身运行时，其对应的进程状态为运行中（R），因为遍历进程的代码正在执行。
 - 所有进程的PID、父PID、状态均正确输出，个人信息（学号、姓名）显示正常，遍历功能实现成功。

3. 常见问题与解决方案

问题现象	原因分析	解决方案
编译时提示“没有规则可制作目标 ‘modules’ ”	未安装linux-headers，或KDIR路径错误	安装对应版本的linux-headers，执行 <code>uname -r</code> 验证内核版本
insmod加载模块时提示“权限不够”	加载内核模块需要root权限	前缀添加 <code>sudo</code> （如 <code>sudo insmod xxx.ko</code> ）
rmmod卸载时提示“模块不存在”	模块名错误，或已卸载	执行`lsmod

进程遍历无输出	内核版本差异导致 <code>for_each_process</code> 宏不可用	确保头文件包含 <code><linux/sched/task.h></code> 和 <code><linux/sched/signal.h></code> ，更新内核或headers版本
---------	--	--

七、实验总结

本次实验成功完成了两个核心目标：

1. 实现了简单Linux内核模块的开发、编译、加载与卸载，理解了内核模块的动态扩展机制，掌握了 `printk`、`jiffies` 等内核工具的使用。
2. 编写了进程遍历模块，利用 `task_struct` 结构体和 `for_each_process` 宏遍历所有进程，成功获取并输出了进程名、PID、父PID、状态等关键信息，熟悉了Linux内核中进程的组织方式。

通过实验，深刻体会到Linux内核模块的灵活性（无需重启系统即可扩展内核功能），同时也认识到内核编程与用户态编程的差异（如不能使用 `printf`、需关注内核API兼容性）。后续可进一步学习内核模块的参数传递、中断处理等高级功能，加深对Linux内核的理解。

附录：实验相关代码清单

1. hellokmodule.c（简单内核模块）

代码块

```
1  #include <linux/init.h>
2  #include <linux/kernel.h>
3  #include <linux/module.h>
4  #include <linux/sched/task.h>
5  #include <linux/sched/signal.h>
6  #include <linux/jiffies.h>
7
8  static int __init hellokmodule_init(void)
9  {
10     printk(KERN_ALERT "ALAL: simple module initialized\n");
11     printk(KERN_INFO "ALAL: Jiffies value: %lu\n", jiffies);
12     return 0;
13 }
14
15 static void __exit hellokmodule_exit(void)
16 {
17     printk(KERN_ALERT "ALAL: simple module is being unloaded\n");
```

```

18     printk(KERN_INFO "ALAL: Jiffies value: %lu\n", jiffies);
19 }
20
21 module_init(hellokmodule_init);
22 module_exit(hellokmodule_exit);
23
24 MODULE_LICENSE("GPL");
25 MODULE_AUTHOR("LKD");
26 MODULE_DESCRIPTION("Simple Kernel Module");
27 MODULE_VERSION("1.01");

```

2. list_processes.c (进程遍历模块)

代码块

```

1  #include <linux/init.h>
2  #include <linux/kernel.h>
3  #include <linux/module.h>
4  #include <linux/sched.h>           // 仅保留核心头文件（你的内核肯定有这个）
5  #include <linux/sched/task.h>
6  #include <linux/sched/signal.h>
7
8  // 直接用数值定义进程状态（替代缺失的TASK_*宏，适配所有内核版本）
9  #define TASK_RUNNING          0x0000
10 #define TASK_INTERRUPTIBLE    0x0001
11 #define TASK_UNINTERRUPTIBLE  0x0002
12 #define TASK_ZOMBIE           0x0004
13 #define TASK_STOPPED          0x0008
14 #define TASK_TRACED           0x0010
15 #define TASK_DEAD             0x0020
16
17 // 进程状态转换函数：用数值匹配，不依赖任何宏
18 static const char* get_process_state(long state)
19 {
20     switch (state) {
21         case TASK_RUNNING:      return "R (运行中)";
22         case TASK_INTERRUPTIBLE: return "S (可中断睡眠)";
23         case TASK_UNINTERRUPTIBLE: return "D (不可中断睡眠)";
24         case TASK_ZOMBIE:       return "Z (僵尸进程)";
25         case TASK_STOPPED:      return "T (停止)";
26         case TASK_TRACED:       return "t (被跟踪)";
27         case TASK_DEAD:         return "X (死亡)";
28         default:                return "Unknown (未知状态)";
29     }

```

```

30 }
31
32 // 模块初始化函数 (遍历进程)
33 static int __init list_processes_init(void)
34 {
35     struct task_struct *task;
36     // 替换为你的学号和姓名
37     printk(KERN_ALERT "==== 进程列表 (学号: 2414308 姓名: 郭威威) ==== \n");
38     printk(KERN_INFO "进程名(comm) | PID | 父进程PID(ppid) | 进程状态 \n");
39     printk(KERN_INFO "----- \n");
40
41     // 遍历所有进程 (for_each_process在linux/sched.h中定义)
42     for_each_process(task) {
43         printk(KERN_INFO "%-12s | %-4d | %-14d | %s \n",
44             task->comm,                // 进程名
45             task_pid_nr(task),          // 安全获取PID
46             task_pid_nr(task->parent),   // 父进程PID
47             get_process_state(task->__state) // 新内核用__state
48         );
49     }
50
51     printk(KERN_ALERT "==== 进程遍历完成 ==== \n");
52     return 0;
53 }
54
55 // 模块卸载函数
56 static void __exit list_processes_exit(void)
57 {
58     printk(KERN_ALERT "进程遍历模块已卸载 \n");
59 }
60
61 // 注册模块入口/出口
62 module_init(list_processes_init);
63 module_exit(list_processes_exit);
64
65 // 模块元信息 (必须)
66 MODULE_LICENSE("GPL");
67 MODULE_AUTHOR("2414308 郭威威");
68 MODULE_DESCRIPTION("List All Processes (Kernel 6.17.1)");
69 MODULE_VERSION("1.2");

```

3. 对应的Makefile (以list_processes为例)

```
1  obj-m := list_processes.o
2  KDIR := /lib/modules/$(shell uname -r)/build
3  PWD := $(shell pwd)
4
5  all:
6      $(MAKE) -C $(KDIR) M=$(PWD) modules
7
8  clean:
9      $(MAKE) -C $(KDIR) M=$(PWD) clean
10
11 testload:
12     sudo dmesg -C
13     sudo insmod list_processes.ko
14     dmesg
15
16 testunload:
17     sudo rmmod list_processes
18     dmesg | grep "进程遍历模块已卸载"
```